Qwen3.5-4B at up to 11,000 tokens/s prefill and 200 tokens/s decode on one AMD GPU

Rotated INT4 on RDNA3: what it is, what it gets you, and how to reproduce it

2 November 2026

The short version

The premise. Four-bit weights are half the size of eight-bit ones, and the matrix units of AMD's RDNA3 GPUs multiply four-bit integers at twice the rate of eight-bit ones. llama.cpp does not use them, because plain four-bit models lose accuracy. We make the accuracy loss small enough to use the hardware: **rotate** the weights and activations so the outliers disappear, **quantise** the weights with GPTQ, and **multiply on the int4 units**.

On the stock **Qwen3.5-4B**, on one Radeon RX 7900 XT, against the standard llama.cpp 4-bit format (Q4_K_M), measured on the same build in the same session:

	prefill (tokens/s)		decode	accuracy vs the BF16 model			VRAM
Qwen3.5-4B, 4-bit	pp512	pp2048	(tokens/s)[†]	KLD ↓	PPL	top-1	peak[‡]
Q4_K_M (standard llama.cpp)	5,477	5,387	147.4	0.028	+2.0%	91.9%	3.62 GB
rotated INT4, `quality` (default)	**7,140**	**7,013**	**202.2**	**0.024**	+1.9%	92.7%	3.36 GB
rotated INT4, `fast` (one setting)	**11,012**	**10,624**	**202.0**	0.097	+8.7%	86.1%	3.36 GB

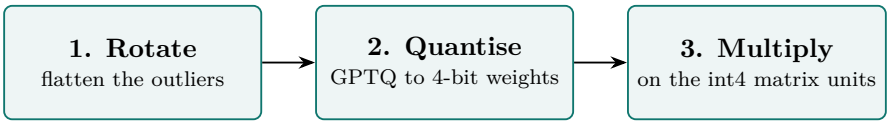
[†]Decode after a 128-token context; from an empty context every row is 1.4–2.2 tokens/s lower and the ratios are the same to within 0.01. [‡]Peak GPU memory during prefill plus decode, measured for `quality`; `fast` uses the same weights.

- `quality` is the same accuracy as Q4_K_M (perplexity +1.9% against +2.0%; slightly lower KLD and higher top-1) at **1.3× the prefill speed and 1.4× the decode speed**, in 7% less memory.
- `fast` is one environment variable away: **2.0× the prefill speed** of Q4_K_M, same decode speed, at a measured accuracy cost (perplexity +8.7%, KLD 0.097).
- **It is not specific to one model.** The same recipe on Ossicle-4B, a fine-tune of the same backbone with an audio encoder, gives the same speed and the same accuracy pattern (Section 3).

How to read the numbers. *Prefill* is reading the prompt (pp512 = a 512-token prompt); *decode* is writing the answer one token at a time. *KLD* is how far the model's output probabilities have moved from the original BF16 model (lower is better); *PPL* is the same comparison as a perplexity change on WikiText-2; *top-1* is how often both models pick the same next token. Accuracy: 24 chunks of WikiText-2 test (about 12,000 tokens), context 512, each model compared

with its own BF16 original; the standard error of a KLD is about ±0.001–0.002 for the first two rows and ±0.003 for `fast`. Speed: median of eight `llama-bench` runs after dropping the first. Q4_K_M was built from the same BF16 file with an importance matrix from WikiText-2 train, like a user would.

1 The idea in three steps



1. Rotate. A language model's activations contain a few channels with huge values. Rounding to four bits ruins them, which is why naive 4-bit models are bad. Multiplying by a Hadamard-style rotation (an orthogonal matrix) before rounding spreads each outlier evenly over 256 values. Because the rotation is orthogonal, rotating the weights *offline* and the activations *at run time* gives exactly the original result before rounding, so nothing is lost by doing it.

2. Quantise. The rotated weights are rounded to symmetric 4-bit integers with GPTQ (error-compensated rounding using 128 calibration sequences of WikiText-2 text), with one scale per 32 weights. The result is stored as a normal-looking GGUF file of the same size as Q4_0 (2.55 GB for this model including a Q6_K output head).

3. Multiply on int4 units. RDNA3 has matrix instructions for 4-bit integers at twice the int8 rate. The weights stay 4-bit; the activations are 8-bit (the default, `quality`: two 4-bit passes) or 4-bit (`fast`: a single pass, twice as fast and less accurate). Decode always uses 8-bit activations.

preset	activations	how to select	what it is for
`quality`	8-bit (two int4 passes)	default	Q4_K_M-level accuracy, faster
`fast`	4-bit (one int4 pass)	`GGML_INT4_PREFILL=fast`	maximum prefill speed, accuracy cost

2 Result: stock Qwen3.5-4B

Figure 1 in three sentences. The default preset keeps accuracy where Q4_K_M has it and is 30% faster at reading the prompt and 37% faster at writing. The `fast` preset doubles the prompt speed and pays for it in accuracy: its KLD is 3.5× Q4_K_M's. Decode speed is the same in both presets because decode does not use the 4-bit-activation path.

What this does not claim. The `quality` advantage in *accuracy* over Q4_K_M is small: perplexity is equal within noise, and the KLD difference (0.024 against 0.028) is only a little larger than the error bars. The claim is "same accuracy, clearly faster", not "more accurate". The `fast` preset is not free.

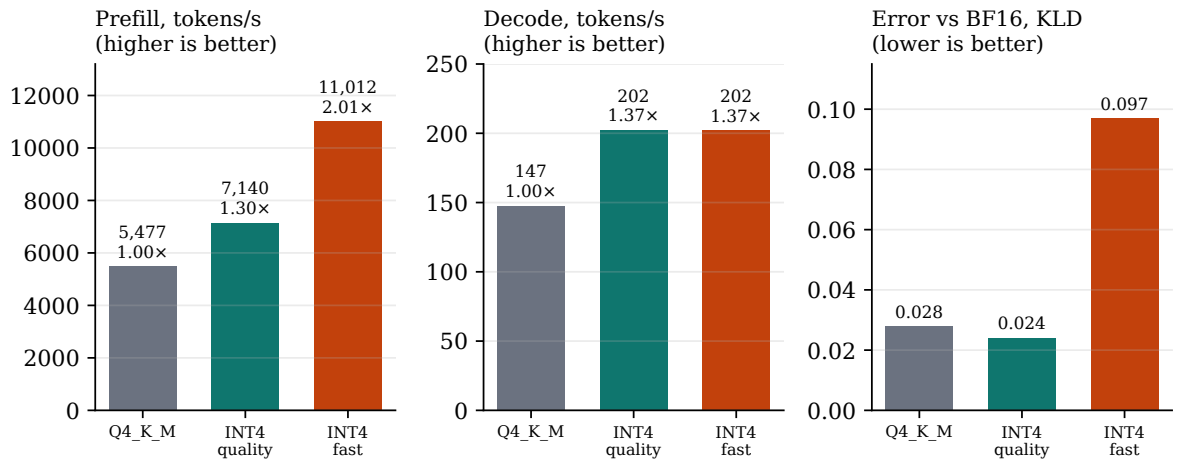


Figure 1: Stock Qwen3.5-4B on one RX 7900 XT. The multipliers are against Q4_K_M. Prefill is pp512; decode is after a 128-token context.

3 It is not Ossicle-specific

This work started on Ossicle-4B, an audio–language model whose language backbone is Qwen3.5-4B. The obvious objection is that the recipe only works for that model. So we ran the *identical pipeline* on the untouched Qwen3.5-4B checkpoint from Hugging Face and measured both models the same way, on the same build:

model	format	pp512	pp2048	decode	KLD	PPL	top-1
Qwen3.5-4B (stock)	Q4_K_M	5,477	5,387	147.4	0.028	+2.0%	91.9%
	rotated INT4 `quality`	7,140	7,013	202.2	0.024	+1.9%	92.7%
	rotated INT4 `fast`	11,012	10,624	202.0	0.097	+8.7%	86.1%
Ossicle-4B (fine-tune)	Q4_K_M	5,461	5,356	147.2	0.031	+3.5%	91.2%
	rotated INT4 `quality`	7,094	6,992	202.0	0.025	+2.0%	92.3%
	rotated INT4 `fast`	10,934	10,606	202.3	0.101	+9.1%	85.0%

Table 1: Both models, one pipeline, one build. Accuracy is each model against its own BF16 original. Same units as the first table.

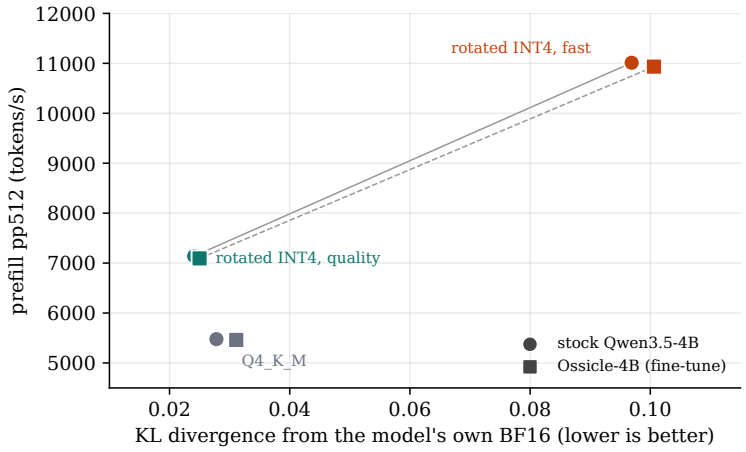


Figure 2: Speed against accuracy for both models. Circles: stock Qwen3.5-4B. Squares: Ossicle-4B. The two models sit on top of each other.

The two models behave the same: speed within 1% (the architecture is identical, so the kernels do the same work), the ordering `quality` better than Q4_K_M and `fast` much faster but less accurate in both, and `fast`'s KLD at 3.2–3.5× Q4_K_M's in both. Nothing in the method is specific to Ossicle: the stock model goes through the same scripts (Section 6); the only change was teaching the GPTQ script to read the original checkpoint's tensor names. Ossicle's extra parts (audio encoder, fine-tuned weights) are not involved in these measurements.

What this does not show. Both models are 4 billion-parameter models of one architecture (Qwen3.5: gated-delta-net layers alternating with full attention). We have *not* measured other model families or sizes. That is the next test, not something this document claims.

4 Why it is faster

Prefill: the int4 units are the fast ones. Prefill is compute-bound. The standard llama.cpp 4-bit path unpacks 4-bit weights to 8-bit and uses the int8 matrix units, which reach about 36–40 TOPS on this card in our tests. Ours feeds the int4 units directly and reaches about 134 int4-TOPS (74% of the hardware's int4 peak): in `quality` mode that is the same work as 67 int8-TOPS, in `fast` mode twice that. We also rewrote the linear-attention and full-attention kernels on the matrix units and fused many small operations into their neighbours.

Decode: stop launching so many kernels. Decode is memory-bound: each token reads the whole 2.5 GB of weights, which at this card's 750 GB/s takes 3.4 ms, a ceiling of about 297 tokens/s. The first version spent over a third of each token idle, launching 847 tiny GPU kernels. Fusing them into 236 raised decode from 138 to 200 tokens/s, now 67% of the ceiling. What remains is about 1 ms per token of gaps between kernel launches.

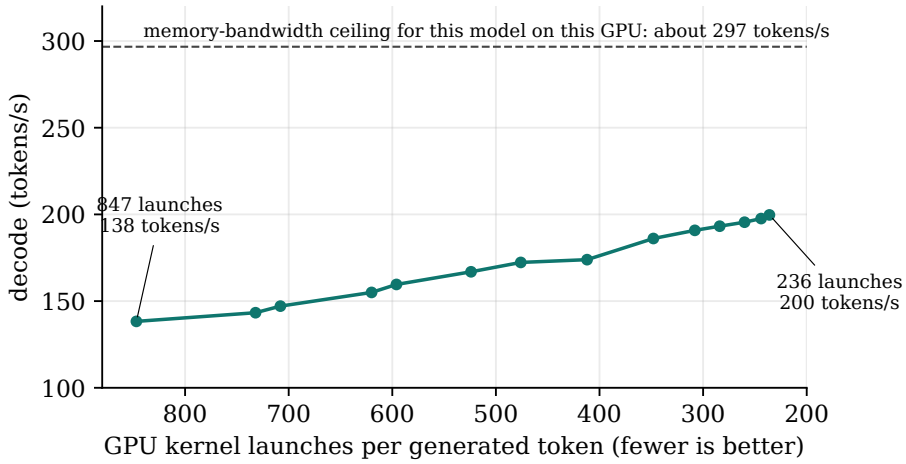


Figure 3: Decode speed against kernel launches per token, through the work (readings from different sessions; decode from an empty context).

5 Why the accuracy holds

- **Rotation makes four bits usable.** Without it, the outlier channels dominate the rounding error. With it, each 256-value block is close to Gaussian, which is what low-bit rounding handles well.
- **The activations decide the accuracy, not the weights.** Four-bit weights with 8-bit activations (`quality`) land at KLD 0.024; changing only the activations to 4-bit (`fast`) takes it to 0.097. Eight bits of activation are enough; four are not, which is why `fast` costs accuracy and `quality` does not.
- **GPTQ and small groups.** In our simulations on Ossicle-4B, GPTQ was worth about 0.03 KLD over plain rounding, and one scale per 32 weights beat one per 64 or 128 by 0.01–0.02 KLD. The technical report has the full ablation.

6 Reproduce it on stock Qwen3.5-4B

You need an RDNA3 GPU (we used an RX 7900 XT, ROCm 10.1.0), about 15 GB of system RAM for the GPTQ step, and about 20 GB of disk, and WikiText-2 (`wiki.train.raw`, `wiki.test.raw`). The GPTQ step itself takes about four minutes.

```
# 0. llama.cpp fork with the INT4 path (branch "ossicle"), built for RDNA3
cmake -S . -B build -DGGML_HIP=ON -DAMDGPU_TARGETS=gfx1100 -DCMAKE_BUILD_TYPE=Release && cmake
    --build build -j

# 1. the unmodified model from Hugging Face -> BF16 GGUF
huggingface-cli download Qwen/Qwen3.5-4B --local-dir qwen35-4b
python convert_hf_to_gguf.py qwen35-4b --outtype bf16 --no-mtp --outfile qwen35-4b-bf16.gguf

# 2. GPTQ on the rotated weights (about 4 minutes, about 15 GB RAM; calibration text = WikiText
    -2 train)
python scripts/gptq_rotate.py --src qwen35-4b --out gptq_g32.safetensors --group 32 --calib 128

# 3. write the GGUF (checks every tensor against the BF16 weights; Q6_K output head)
python scripts/rotate_quantize.py qwen35-4b-bf16.gguf qwen35-4b-rotq4-gptq.gguf --blocks
    gptq_g32.safetensors --embd q6_k
```

```
# 4. run it
llama-server -m qwen35-4b-rotq4-gptq.gguf -ngl 99                              # quality (default)
GGML_INT4_PREFILL=fast llama-server -m qwen35-4b-rotq4-gptq.gguf -ngl 99     # fast

# 5. check the numbers in this document
llama-bench -m qwen35-4b-rotq4-gptq.gguf -ngl 99 -p 512,2048 -n 128 -d 0,128 -r 9
llama-perplexity -m qwen35-4b-bf16.gguf -f wiki.test.raw -c 512 -ngl 99 --chunks 24 --kl-
    divergence-base base.bin   # saves the BF16 reference
llama-perplexity -m qwen35-4b-rotq4-gptq.gguf -f wiki.test.raw -c 512 -ngl 99 --chunks 24 --kl-
    divergence-base base.bin --kl-divergence
```

The Q4_K_M baseline in this document was made from the same BF16 file with `llama-imatrix` (WikiText-2 train, 250 chunks of 1024 tokens) and `llama-quantize` with that importance matrix, and measured with the same commands.

7 Limits

- **RDNA3 only.** The speed comes from RDNA3's int4 matrix instructions (RX 7000 series; we tested one RX 7900 XT). Other GPUs would need their own kernels.
- **Not an ordinary GGUF.** The files look like Q4_0 but hold rotated codes; any build other than this branch decodes them as garbage. The GGUF carries a key that the branch detects.
- **One architecture, one size.** Two models, both 4B Qwen3.5. Other families and sizes are untested.
- **`fast` is a trade.** 2× prefill for KLD 0.097 against 0.024; choose per use case. We do not call it equivalent to `quality`.
- **Text calibration.** GPTQ used WikiText-2 text only; the accuracy numbers are text numbers.
- **Measurement noise.** A decode figure moves by about ±1.5 tokens/s with GPU temperature, which is why the comparisons are against Q4_K_M measured in the same session. The first prefill sample of a run is slow, so the table uses the median after dropping it.
- **Untested:** multi-GPU, loading and unloading a model in one process, long contexts for the new attention path beyond the figures in the technical report.

Where everything is

Code: the `ossicle` branch of the llama.cpp fork (`SILVERTHRONE32/llama.cpp-ossicle`); files: `SILVERTHRONE/Ossicle-4B-GGUF`, with the stock-model file at `rdna3/qwen35-4b-rotq4-gptq.gguf`. Licence: Apache 2.0. Every experiment, including the failed ones, is in the full technical report (*Q4-quality INT4 inference for Ossicle-4B on RDNA3*), which this document summarises.